

# CS3700 Lab 2: Generalization and Specialization

**Name:** Utsav Acharya

**Date:** September 22, 2026

## Case Study 1: Campus Parking

**Generalization or Specialization:** Specialization

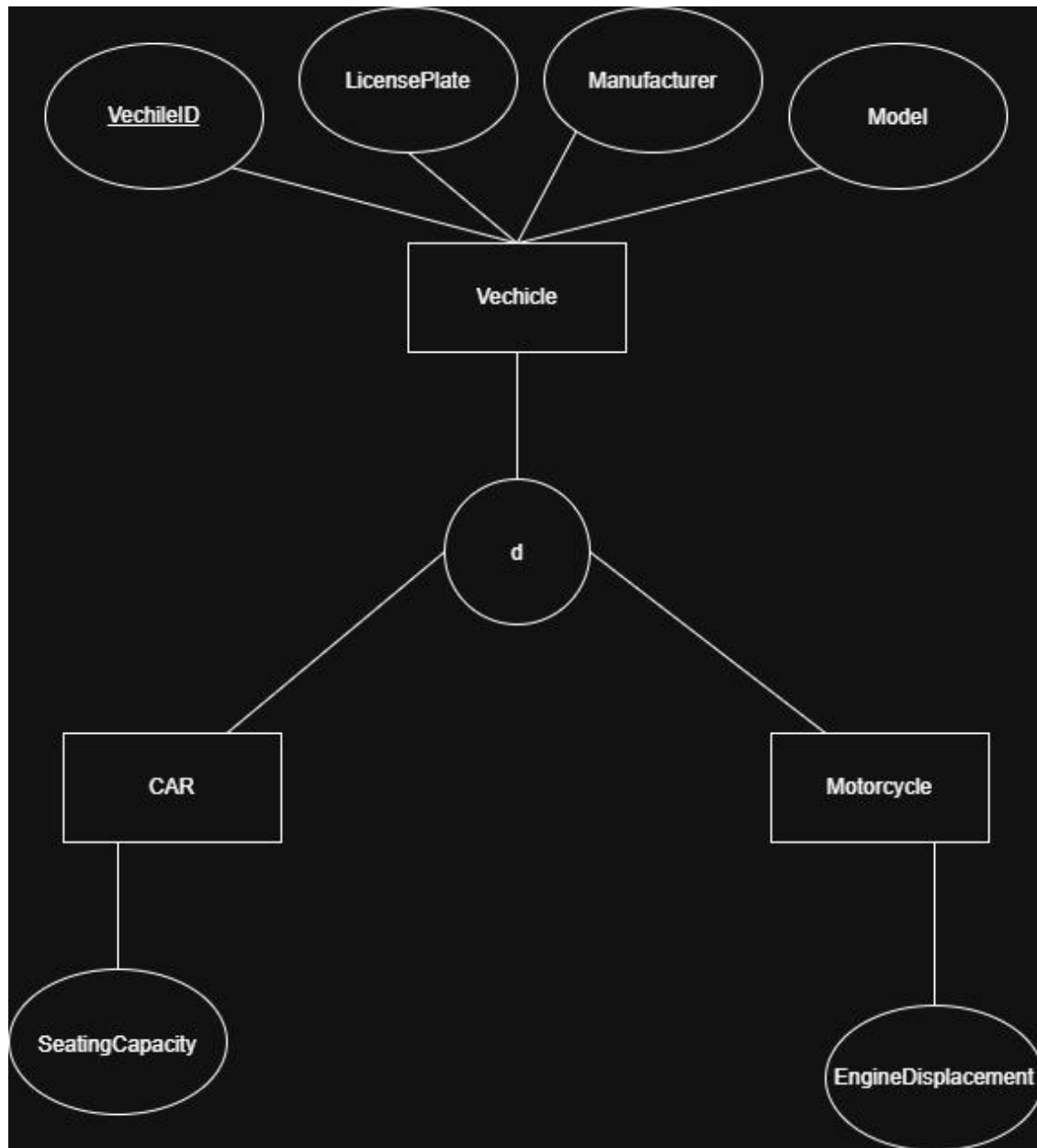
**Explanation:** Vehicle already exists as a single entity, and we are working top-down to carve out Car and Motorcycle to capture attributes that only apply to those specific types. Because we start from an existing general entity and derive more specific types from it, this is specialization rather than generalization.

### Superclass, Subclasses, and Attributes

- Superclass: Vehicle (VehicleID [key], LicensePlate, Manufacturer, Model)
- Subclass - Car: SeatingCapacity
- Subclass - Motorcycle: EngineDisplacement

Note: Both subclasses inherit VehicleID, LicensePlate, Manufacturer, and Model from Vehicle; these are not repeated in the subclasses.

ERD



d=disjoint, Single line = partial participation

## Constraint Justification

- Disjoint (d): The business rules state a registered vehicle cannot be both a car and a motorcycle, so the specialization is disjoint (d)—each Vehicle belongs to at most one of the two subclasses.
- Partial (Single Line): The system also registers other vehicles, such as buses, that are not cars or motorcycles and carry no extra type-specific attributes. Since not every Vehicle instance falls into a subclass, participation is partial (single line).

## Discussion Question

*Why might adding both new attributes directly to Vehicle be an unsuitable design?*

Adding SeatingCapacity and EngineDisplacement directly to Vehicle would force every row to carry both attributes even though each one only makes sense for one vehicle type. Cars would have a meaningless EngineDisplacement value and motorcycles a meaningless SeatingCapacity value, producing unnecessary null values and wasted storage for buses and other vehicles that use neither attribute. It would also make it easier to accidentally record both values for the same vehicle, violating the business rule that a vehicle cannot be both a car and a motorcycle.

Modeling Car and Motorcycle as subclasses keeps each attribute attached only to the vehicle type it actually describes.

## Case Study 2: University Contacts Generalization or Specialization

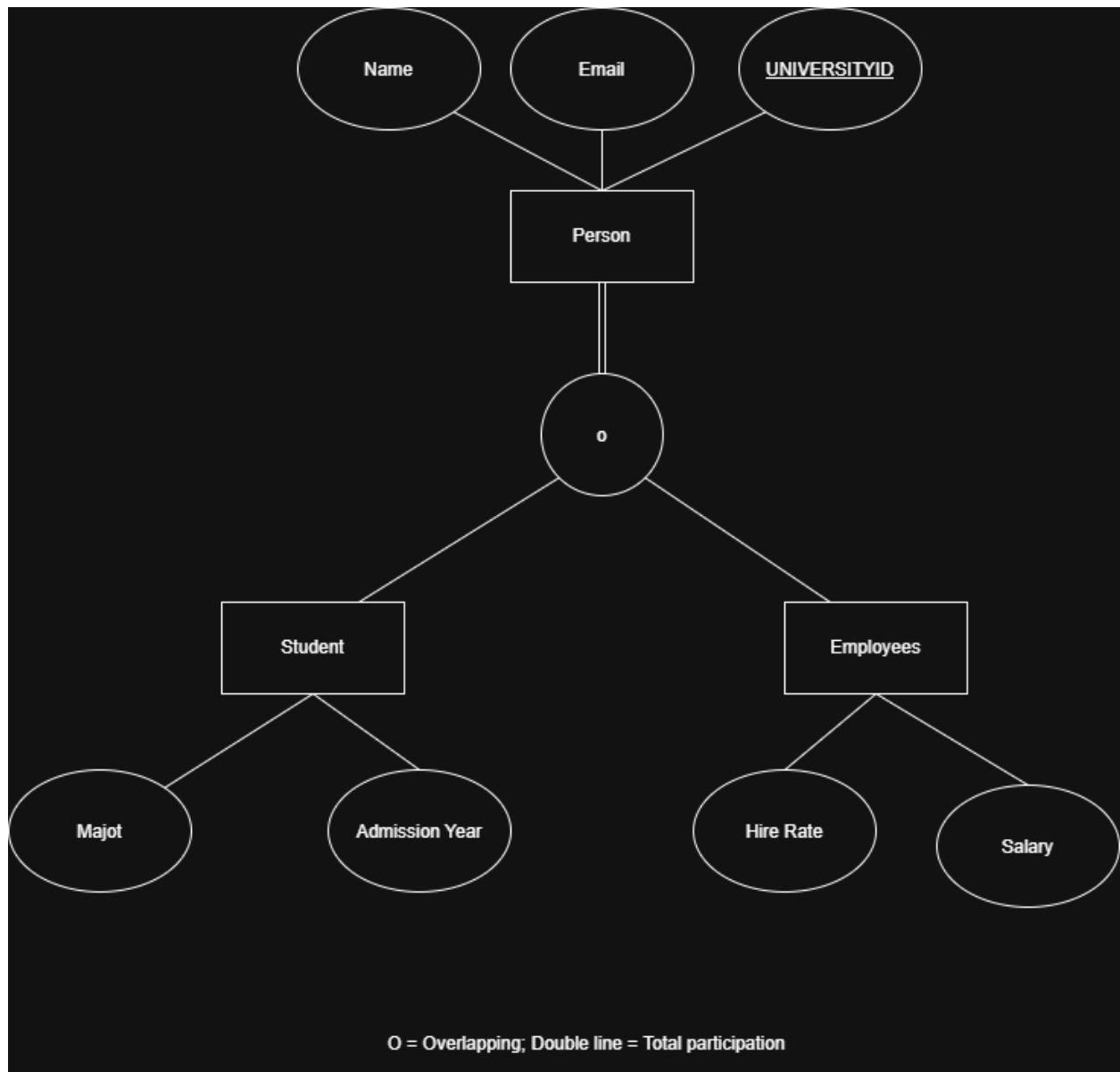
**Decision:** Generalization

**Explanation:** Student and Employee already exist as two separate entities maintained by different offices. We are working bottom-up, factoring out their shared attributes (UniversityID, Name, Email) into a new superclass, Person. Because we start from existing specific entities and combine them upward into a more general entity, this is generalization.

- Superclass, Subclasses, and Attributes
- Superclass: Person (UniversityID [key], Name, Email)
- Subclass - Student: Major, AdmissionYear
- Subclass - Employee: HireDate, Salary

Note: Both subclasses inherit UniversityID, Name, and Email from Person; these are not repeated in the subclasses.

## ERD



## Constraint Justification

- Overlapping (o): The business rules state a person may be both a student and an employee, so the same Person instance can participate in both subclasses at once—the specialization is overlapping (o).
- Total (Double Line): The combined system will contain only people who are students, employees, or both, meaning every Person instance must belong to at least one subclass. Participation is therefore total (double line).

### **Discussion Question**

*How should the model represent someone who is both a student and an employee without duplicating their shared information?*

The person is stored as a single Person row, identified once by UniversityID, holding the shared Name and Email attributes exactly once. That same UniversityID then also appears as a row in both the Student subclass (with Major and AdmissionYear) and the Employee subclass (with HireDate and Salary). Because the overlapping generalization lets one superclass instance participate in more than one subclass, the shared information lives in Person and is never copied into either subclass.

## **Case Study 3: Library Resources**

## Generalization or Specialization

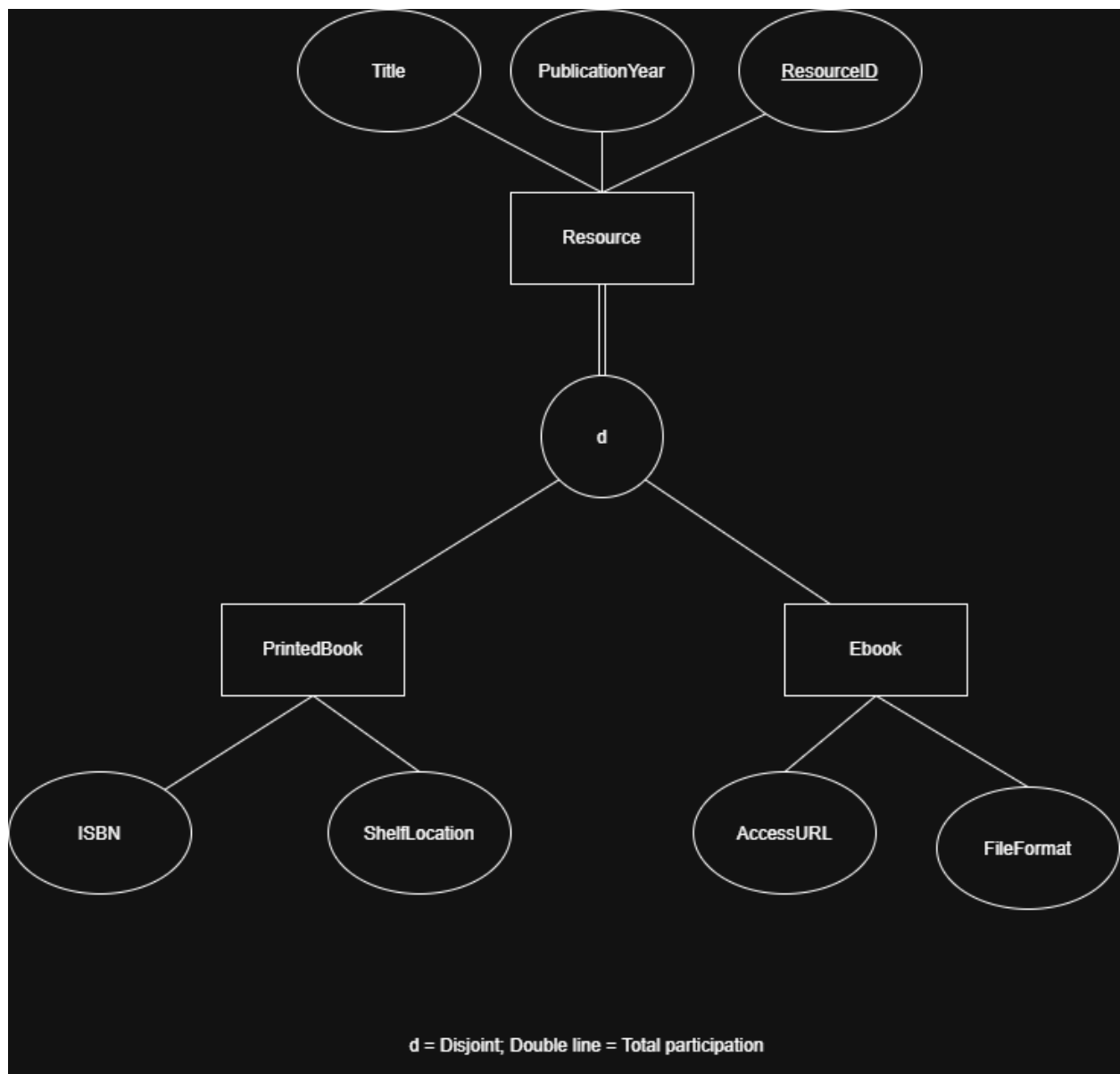
**Decision:** Specialization

**Explanation:** Resource already exists as a single entity, and we are working top-down to derive PrintedBook and Ebook to capture attributes specific to each format. Because we start from an existing general entity and derive more specific types from it, this is specialization.

### Superclass, Subclasses, and Attributes

- Superclass: Resource (ResourceID [key], Title, PublicationYear)
- Subclass - PrintedBook: ISBN, ShelfLocation
- Subclass - Ebook: AccessURL, FileFormat

Note: Both subclasses inherit ResourceID, Title, and PublicationYear from Resource; these are not repeated in the subclasses.



## Constraint Justification

- **Disjoint (d):** Every resource must belong to exactly one of the two categories, and a title available in both formats receives a separate ResourceID for each format. Since no single ResourceID instance is ever both a PrintedBook and an Ebook, the specialization is disjoint (d).
- **Total (Double Line):** The library tracks only printed books and e-books, so every Resource instance must fall into one of the two subclasses. Participation is therefore total (double line).

## Discussion Question

*Does having the same title in both formats make the subclasses overlapping? Explain using the resource identity rule.*

No. Overlap in a specialization is about whether the same entity instance belongs to more than one subclass, not about whether two instances share a descriptive attribute value like Title. The business rule states that a title available in both formats receives a separate ResourceID for each format, so the printed version and the e-book version are two distinct Resource instances, each belonging to exactly one subclass. Because identity is defined by ResourceID rather than Title, the specialization remains disjoint even though the two records describe the same work.

## Case Study 4: Equipment Inventory

### Generalization or Specialization

**Decision:** Generalization

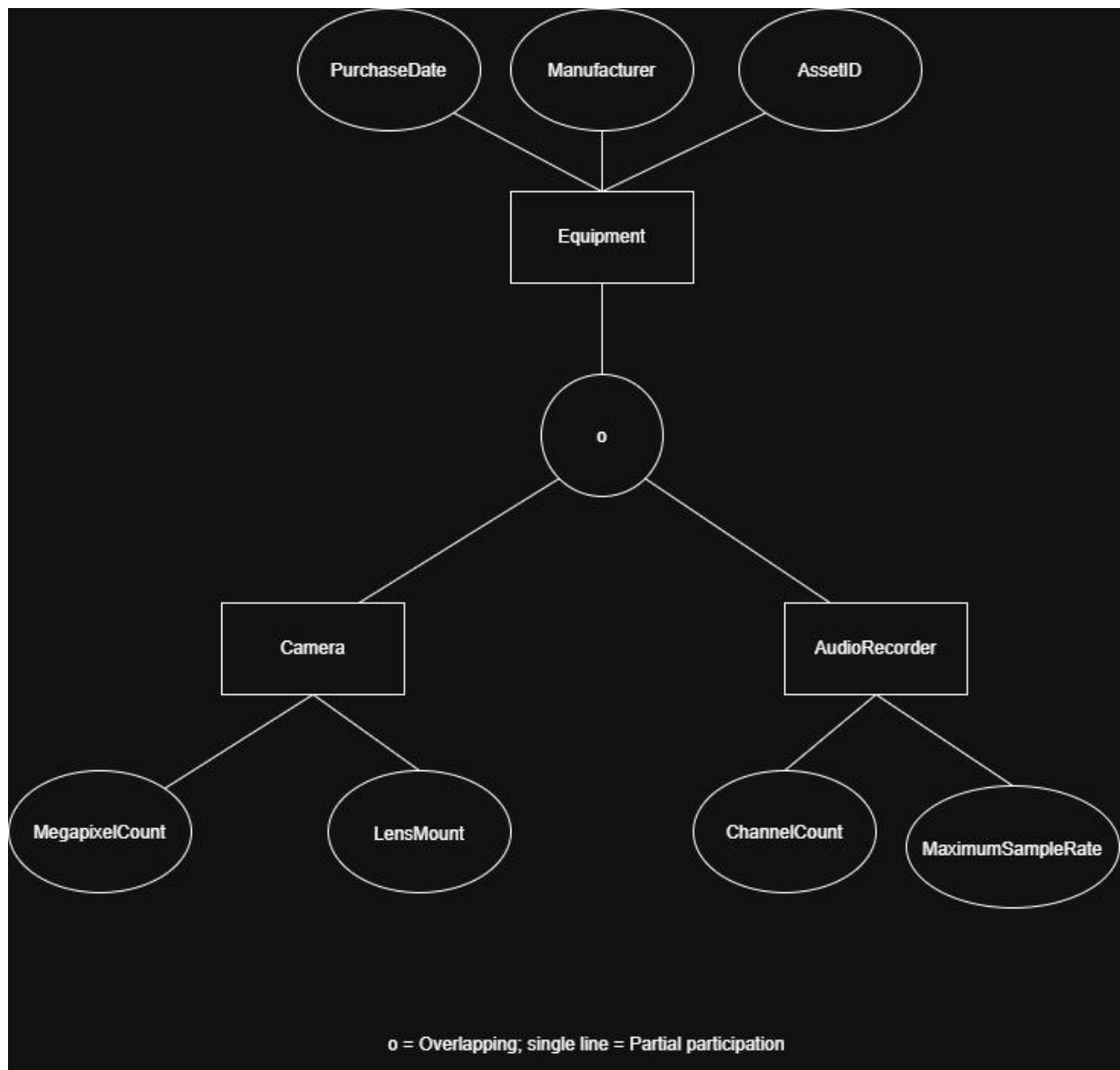
**Explanation:** Camera and AudioRecorder already exist as two separate equipment records. We are working bottom-up, factoring out their shared attributes (AssetID, Manufacturer, PurchaseDate) into a new superclass, Equipment. Because we start from existing specific entities and combine them upward into a more general entity, this is generalization.

### Superclass, Subclasses, and Attributes

- Superclass: Equipment (AssetID [key], Manufacturer, PurchaseDate)
- Subclass - Camera: MegapixelCount, LensMount
- Subclass - AudioRecorder: ChannelCount, MaximumSampleRate

Note: Both subclasses inherit AssetID, Manufacturer, and PurchaseDate from Equipment; these are not repeated in the subclasses.

ERD



**Constraint Justification**

- Overlapping (o): Some hybrid devices qualify as both cameras and audio recorders and require both sets of type-specific attributes, so the same Equipment instance can participate in both subclasses at once—the specialization is overlapping (o).
- Partial (Single Line): The inventory also includes lighting equipment, for which only the shared attributes are recorded and which is not a Camera or an AudioRecorder. Since not every Equipment instance falls into a subclass, participation is partial (single line).

## Discussion Question

*Where should the model store a hybrid device's Manufacturer and PurchaseDate?*

Manufacturer and PurchaseDate are shared attributes, so they are stored once in the Equipment superclass, never duplicated in Camera or AudioRecorder. A hybrid device is represented as a single Equipment row (identified by AssetID) that also has a corresponding row in the Camera subclass (for MegapixelCount and LensMount) and a corresponding row in the AudioRecorder subclass (for ChannelCount and MaximumSampleRate), all linked by the same AssetID. The overlapping generalization is exactly what allows one Equipment instance to extend into both subclasses without repeating its shared attributes.